KU LEUVEN

Labo 5 - CUDA

Dynamic Parallelism en Asynchronous Operation

Sander De Stercke (MEIECFS)
Isaak Vervenne (MEISWE)

Maart 2026

1 Wat is er gerealiseerd?

Deze labo-sessie heeft als doel het vergelijken van zowel een synchrone als een asynchrone uitvoering, om zo de efficiëntie ten opzichte van elkaar te bepalen. We hebben functionaliteiten geïmplementeerd om zowel de som, het product, het minimum en het maximum van 4 arrays te berekenen (uitbreiding op opgave 2). Deze operaties worden op 4 onafhankelijke arrays uitgevoerd en worden verwerkt op de GPU.

De totale uitvoeringstijd wordt berekend inclusief de CPU verwerkingstijd, en daarna vergeleken. Om zo een eerlijker beeld te krijgen op beide methoden.

Er is een workflow opgezet waarbij data op de CPU wordt gegenereerd en vervolgens naar de GPU wordt gemigreerd voor berekening. Dit hebben we op twee verschillende manieren gedaan:

- **Synchroon:** De operaties (array genereren, kopiëren met `cudaMemcpy`, en kernel executie) worden strikt na elkaar uitgevoer. De CPU wacht telkens tot de GPU klaar is.
- **Asynchroon:** We gebruiken `cudaMemcpyAsync` en CUDA-streams om de data-overdracht en kernel-executie op de GPU uit te voeren zonder de CPU te blokkeren. Hierdoor genereren we de volgende array op de CPU, terwijl de GPU de vorige array asynchroon verwerkt.

2 Problemen

2.1 Verlies van precisie (Fout bij grote N)

Uit onze console-output bleek dat de verificatie van de `SUM` operatie mislukt bij zeer grote arrays ($N \geq 16.777.216$). Dit is geen logische fout in onze reductie-code, maar een gekend probleem met de precisie van 32-bit `float` datatypes (single precision). Naarmate de opgeslagen som groter wordt, gaan de kleinste significante bits bij optellingen verloren door afrondingsfouten. Omdat we op de CPU sequentieel opbouwen en op de GPU hiërarchisch (per block), propageren deze afrondingsfouten anders, wat bij miljoenen elementen resulteert in een verschil dat onze tolerantie overschrijdt.

2.2 Correcte tijdmeting (Synchronisatie)

Omdat `cudaMemcpyAsync` en de kernel call (`foo«<...»>`) asynchroon zijn en onmiddellijk terugkeren naar de CPU-code, was het essentieel om handmatig te synchroniseren. Om de correcte totale executietijd te meten, moesten we ervoor zorgen dat we pas de timer stopten ná een `cudaDeviceSynchronize` call aan het einde van de pipeline. Daarnaast was het alloceren van 'pinned memory' via `cudaMallocHost` belangerijk, anders negeert

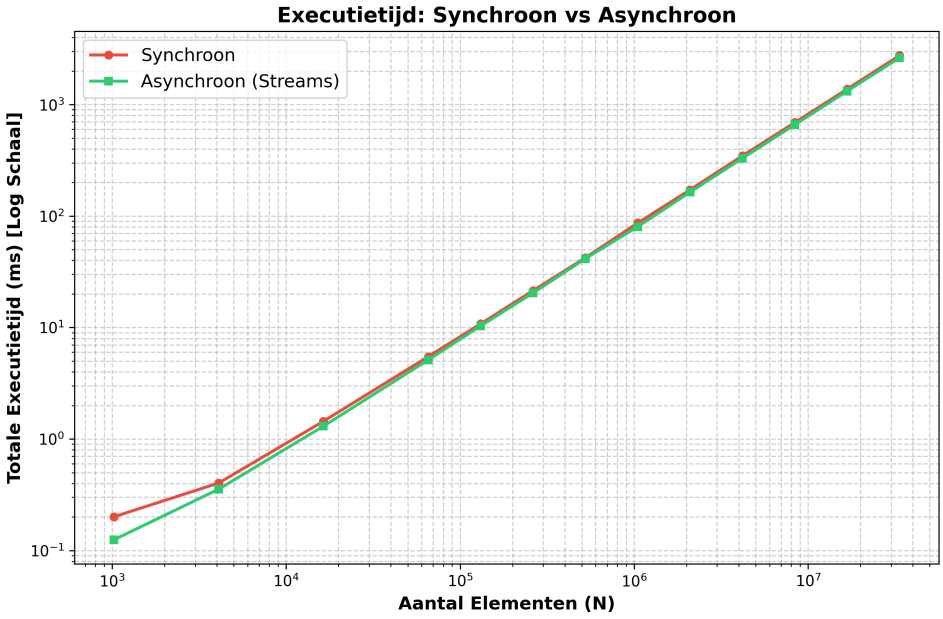
de driver de asynchrone overdracht.

3 Vragen en Analyse

3.1 Vergelijking van de executietijd: Synchroon vs. Asynchroon

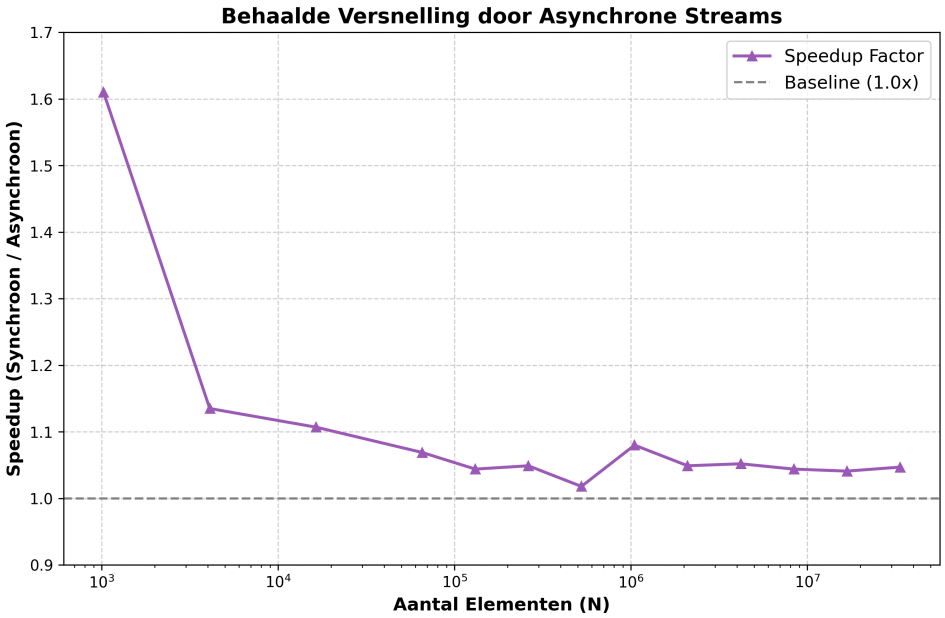
Het primaire doel is het vergelijken van de totale executietijd tussen de synchrone en asynchrone aanpak over verschillende groottes van `N`.

Zoals te zien in Figuur 1, is de asynchrone implementatie consequent sneller. Bij kleine arrays ($N = 1024$) zien we zelfs een grote versnelling (1.6x tot 1.8x speedup). Dit komt omdat de overhead van het wachten (CPU blocking) hier de grootste bottleneck vormt.



Figuur 1: Totale executietijd van de 4 bewerkingen, synchroon versus asynchroon.

Naarmate `N` groeit naar bv. miljoenen elementen, wordt het rekenwerk en de geheugenbandbreedte zelf de belangerijkste factor, niet het opstarten ervan. Toch zien we in Figuur 2 dat we ook bij deze heel grote datasets ($N = 33.554.432$) een stabiele prestatiewinst behouden van ongeveer 4% tot 8% (een speedup factor van ~ 1.05). Door de CPU data te laten genereren terwijl de GPU de voorgaande streams verwerkt, elimineren we de CPU wachttijd nagenoeg volledig. Het dalen van de speedup-factor bij grote N is ook te verklaren doordat de CPU een groter aandeel heeft in de totale executietijd door de extreem grote waarden van N (die gelijk is in beide gevallen). Waardoor het verschil van tussen asynchroon en synchroon minder merkbaar is.



Figuur 2: Relatieve versnelling (speedup) verkregen door het gebruik van asynchrone CUDA-streams.

4 Bijlage: Cuda Code

```cpp
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#include <iostream>
#include <vector>
#include <iomanip>
#include <cmath>
#include <chrono>
#include <numeric>
#include <cstdlib>

const int BLOCK_SIZE = 256;
const int DEFAULT_RUNS = 5; // Standaard aantal herhalingen voor gemiddelden

// ============================================================
// DEFINITIES VOOR OPERATIES
// ============================================================
enum Operation { SUM = 0, PROD = 1, MIN_OP = 2, MAX_OP = 3 };

// ============================================================
// CPU VERIFICATIE EN DATA GENERATIE FUNCTIES
// ============================================================

// Genereert data op de CPU. Product krijgt kleine waarden om overflow te vermijden.
void generateData(float* data, int N, Operation op) {
    for (int i = 0; i < N; ++i) {
        if (op == PROD) {
            data[i] = ((float)rand() / RAND_MAX) * 0.9f + 0.1f; // Tussen 0.1 en 1.0
        } else {
            data[i] = ((float)rand() / RAND_MAX) * 100.0f;
        }
    }
}

// Verifieer GPU output door de volledige array op de CPU te reduceren
bool verifyReduction(const float* input, float gpu_result, int N, Operation op, float
     tolerance = 1e-2) {
    float cpu_result;
    if (op == SUM) {
        cpu_result = 0.0f;
        for (int i = 0; i < N; ++i) cpu_result += input[i];
    } else if (op == PROD) {
        cpu_result = 1.0f;
        for (int i = 0; i < N; ++i) cpu_result *= input[i];
    } else if (op == MIN_OP) {
        cpu_result = INFINITY;
        for (int i = 0; i < N; ++i) cpu_result = std::min(cpu_result, input[i]);
    } else if (op == MAX_OP) {
        cpu_result = -INFINITY;
        for (int i = 0; i < N; ++i) cpu_result = std::max(cpu_result, input[i]);
    }

    // Voor som en product kan float afronding meespelen bij miljoenen elementen
    if (op == MIN_OP || op == MAX_OP) return cpu_result == gpu_result;

    // Relatieve fout gebruiken voor floats
    float diff = std::abs(cpu_result - gpu_result);
    return (diff / std::max(1.0f, std::abs(cpu_result))) < tolerance;
}

```

```
59 // ============================================================
60 // DEEL 1: REDUCTION KERNEL (MET TEMPLATES)
61 // ============================================================
62
63 template <Operation OP>
64 __global__ void reduction_kernel(const float* input, float* output, int N) {
65     extern __shared__ float sdata[];
66
67     unsigned int tid = threadIdx.x;
68     unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
69
70     // Inladen met neutrale elementen voor out-of-bounds draden
71     float val;
72     if (OP == SUM)       val = (i < N) ? input[i] : 0.0f;
73     else if (OP == PROD) val = (i < N) ? input[i] : 1.0f;
74     else if (OP == MIN_OP)  val = (i < N) ? input[i] : INFINITY;
75     else if (OP == MAX_OP)  val = (i < N) ? input[i] : -INFINITY;
76
77     sdata[tid] = val;
78     __syncthreads();
79
80     // Reductie in shared memory
81     for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
82         if (tid < s) {
83             if (OP == SUM)       sdata[tid] += sdata[tid + s];
84             else if (OP == PROD) sdata[tid] *= sdata[tid + s];
85             else if (OP == MIN_OP)  sdata[tid] = fminf(sdata[tid], sdata[tid + s]);
86             else if (OP == MAX_OP)  sdata[tid] = fmaxf(sdata[tid], sdata[tid + s]);
87         }
88         __syncthreads();
89     }
90
91     if (tid == 0) output[blockIdx.x] = sdata[0];
92 }
93
94 // Helper functie om de juiste kernel te lanceren
95 void launch_kernel(Operation op, const float* d_in, float* d_out, int N, dim3 grid,
       dim3 threads, cudaStream_t stream = 0) {
96     size_t shared_mem = threads.x * sizeof(float);
97     if (op == SUM)       reduction_kernel<SUM><<<grid, threads, shared_mem, stream
           >>>(d_in, d_out, N);
98     else if (op == PROD) reduction_kernel<PROD><<<grid, threads, shared_mem, stream
           >>>(d_in, d_out, N);
99     else if (op == MIN_OP)  reduction_kernel<MIN_OP><<<grid, threads, shared_mem,
           stream>>>(d_in, d_out, N);
100    else if (op == MAX_OP)  reduction_kernel<MAX_OP><<<grid, threads, shared_mem,
           stream>>>(d_in, d_out, N);
101 }
102
103 // CPU helper voor de laatste reductiestap (van de blok-output naar 1 resultaat)
104 float final_cpu_reduction(const std::vector<float>& block_results, int num_blocks,
       Operation op) {
105     float result;
106     if (op == SUM) {
107         result = 0.0f;
108         for (int i = 0; i < num_blocks; ++i) result += block_results[i];
109     } else if (op == PROD) {
110         result = 1.0f;
111         for (int i = 0; i < num_blocks; ++i) result *= block_results[i];
112     } else if (op == MIN_OP) {
113         result = INFINITY;
```

```
114         for (int i = 0; i < num_blocks; ++i) result = std::min(result, block_results[
                i]);
115     } else if (op == MAX_OP) {
116         result = -INFINITY;
117         for (int i = 0; i < num_blocks; ++i) result = std::max(result, block_results[
                i]);
118     }
119     return result;
120 }
121 
122 // ============================================================
123 // HOST CODE (CPU)
124 // ============================================================
125 
126 int main(int argc, char** argv) {
127     srand(42);
128 
129     // Aantal herhalingen (standaard DEFAULT_RUNS, of via command-line argument)
130     int num_runs = DEFAULT_RUNS;
131     if (argc >= 2) {
132         num_runs = atoi(argv[1]);
133         if (num_runs < 1) num_runs = 1;
134     }
135 
136     // Veel verschillende array-groottes voor mooie grafieken
137     std::vector<int> test_sizes = {
138         1 << 10,   // 1K
139         1 << 12,   // 4K
140         1 << 14,   // 16K
141         1 << 16,   // 64K
142         1 << 17,   // 128K
143         1 << 18,   // 256K
144         1 << 19,   // 512K
145         1 << 20,   // 1M
146         1 << 21,   // 2M
147         1 << 22,   // 4M
148         1 << 23,   // 8M
149         1 << 24,   // 16M
150         1 << 25,   // 32M
151     };
152 
153     Operation ops[4] = {SUM, PROD, MIN_OP, MAX_OP};
154     std::string op_names[4] = {"Sum", "Product", "Minimum", "Maximum"};
155 
156     std::cout << "Aantal␣runs␣per␣meting:␣" << num_runs << "\n";
157     std::cout << "Aantal␣test-groottes:␣" << test_sizes.size() << "\n\n";
158 
159     // CSV header voor eenvoudige data-import in Python/Excel
160     std::cout << "CSV_START\n";
161     std::cout << "N,Sync_ms,Async_ms,Speedup\n";
162 
163     for (int N : test_sizes) {
164         int num_blocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
165         size_t bytes = (size_t)N * sizeof(float);
166         size_t out_bytes = (size_t)num_blocks * sizeof(float);
167 
168         dim3 threads(BLOCK_SIZE);
169         dim3 grid(num_blocks);
170 
171         // Alloceer geheugen (pinned + device)
172         float *h_data[4], *d_data[4], *d_out[4];
```

```
173         std::vector<float> h_block_results(num_blocks);
174
175         for (int i = 0; i < 4; ++i) {
176             cudaMallocHost(&h_data[i], bytes);
177             cudaMalloc(&d_data[i], bytes);
178             cudaMalloc(&d_out[i], out_bytes);
179         }
180
181         // Streams voor async
182         cudaStream_t streams[4];
183         for (int i = 0; i < 4; ++i) cudaStreamCreate(&streams[i]);
184
185         std::vector<double> sync_times(num_runs);
186         std::vector<double> async_times(num_runs);
187
188         for (int run = 0; run < num_runs; ++run) {
189             // ============ SYNCHRONE UITVOERING ============
190             auto start_sync = std::chrono::high_resolution_clock::now();
191
192             for (int i = 0; i < 4; ++i) {
193                 generateData(h_data[i], N, ops[i]);
194                 cudaMemcpy(d_data[i], h_data[i], bytes, cudaMemcpyHostToDevice);
195                 launch_kernel(ops[i], d_data[i], d_out[i], N, grid, threads);
196                 cudaMemcpy(h_block_results.data(), d_out[i], out_bytes,
                        cudaMemcpyDeviceToHost);
197                 float final_val = final_cpu_reduction(h_block_results, num_blocks,
                        ops[i]);
198                 // Verificatie alleen bij eerste run
199                 if (run == 0) {
200                     bool correct = verifyReduction(h_data[i], final_val, N, ops[i]);
201                     if (!correct) {
202                         std::cerr << "FOUT:␣" << op_names[i] << "␣mislukt␣bij␣N=" <<
                                N << "\n";
203                     }
204                 }
205             }
206
207             auto end_sync = std::chrono::high_resolution_clock::now();
208             sync_times[run] = std::chrono::duration<double, std::milli>(end_sync -
                    start_sync).count();
209
210             // ============ ASYNCHRONE UITVOERING ============
211             auto start_async = std::chrono::high_resolution_clock::now();
212
213             generateData(h_data[0], N, ops[0]);
214
215             for (int i = 0; i < 4; ++i) {
216                 cudaMemcpyAsync(d_data[i], h_data[i], bytes, cudaMemcpyHostToDevice,
                        streams[i]);
217                 launch_kernel(ops[i], d_data[i], d_out[i], N, grid, threads, streams[
                        i]);
218                 cudaMemcpyAsync(d_out[i], d_out[i], out_bytes,
                        cudaMemcpyDeviceToDevice, streams[i]);
219
220                 if (i + 1 < 4) {
221                     generateData(h_data[i + 1], N, ops[i + 1]);
222                 }
223             }
224
225             cudaDeviceSynchronize();
226
```

```
227             auto end_async = std::chrono::high_resolution_clock::now();
228             async_times[run] = std::chrono::duration<double, std::milli>(end_async -
                    start_async).count();
229         }
230
231         // Gemiddelde berekenen
232         double avg_sync = std::accumulate(sync_times.begin(), sync_times.end(), 0.0)
                / num_runs;
233         double avg_async = std::accumulate(async_times.begin(), async_times.end(),
                0.0) / num_runs;
234         double speedup = avg_sync / avg_async;
235
236         // CSV-rij
237         std::cout << N << ","
238                   << std::fixed << std::setprecision(4) << avg_sync << ","
239                   << avg_async << ","
240                   << std::setprecision(3) << speedup << "\n";
241
242         // Detailed output naar stderr (zodat CSV op stdout schoon blijft)
243         std::cerr << "N=" << std::setw(10) << N
244                   << "␣␣Sync:␣" << std::fixed << std::setprecision(4) << avg_sync <<
                        "␣ms"
245                   << "␣␣Async:␣" << avg_async << "␣ms"
246                   << "␣␣Speedup:␣" << std::setprecision(3) << speedup << "x"
247                   << "␣␣(avg␣over␣" << num_runs << "␣runs)\n";
248
249         // Cleanup voor deze grootte
250         for (int i = 0; i < 4; ++i) {
251             cudaFreeHost(h_data[i]);
252             cudaFree(d_data[i]);
253             cudaFree(d_out[i]);
254             cudaStreamDestroy(streams[i]);
255         }
256     }
257
258     std::cout << "CSV_END\n";
259
260     return 0;
261 }
```

Listing 1: L5a.cu - Pseudocode asynchrone implementatie met streams